

A Deep Look Into the Temporal I/O Behavior of HPC Applications*

Francieli Boito, **Luan Teylo**, Mihail Popov, Theo Jolivel, François Tessier, Jakob Luetzgau, Julien Monniot, Ahmad Tarraf, André Carneiro, and Carla Osthoff

** Accepted for publication at IPDPS 2025.*

I/O optimization techniques rely on assumptions about application characteristics. For instance:

- Read at the beginning and write at the end
- Perform I/O periodically
- The PFS experiences a lot of concurrent access

We study these assumptions using ~440K I/O traces from multiple HPC systems.

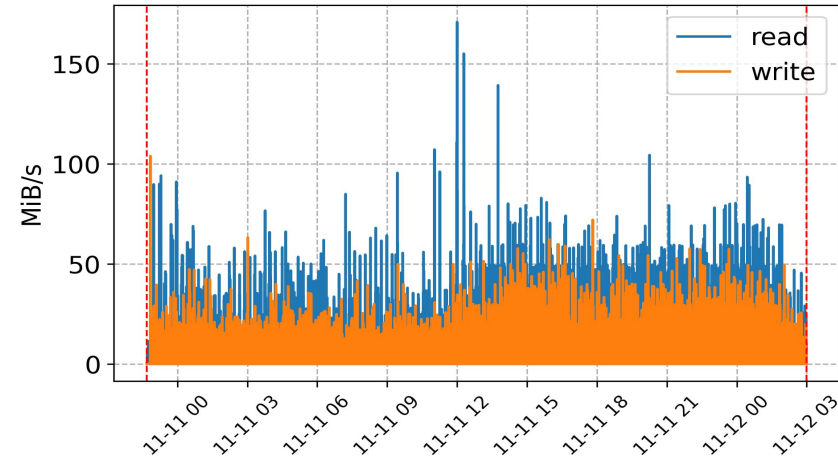
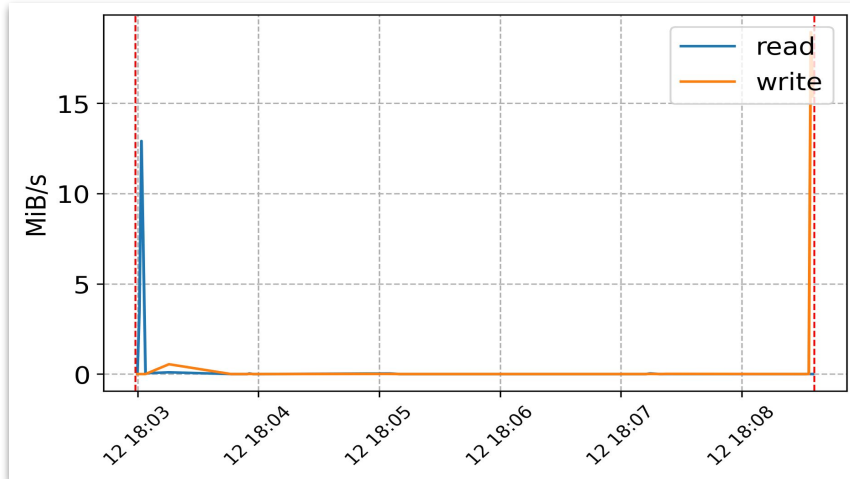
System	Institution	Nodes	File System	Period	Traces Analyzed
Plafrim	Inria - Univ. Bordeaux	192	Beegfs 2 OSS/ 8 OSTs	May 2022 -July 2024 (26 Mo)	16,474 Jobs
Sdumon	LNCC, Brazil	1,134	Lustre 10 OSS / 10 OSTs	1 year (2020)	65,714 Jobs
Blue Waters	University of Illinois	26,000	Lustre 360 OSS / 1,440 OSTs	1 year (2019)	316,399 Jobs
Intrepid	Argonne National Lab	40,960	PVFS & GPFS 128 OSS	1 year (2013)	42,066 Jobs

File System Traces
(time series)

Darshan Traces

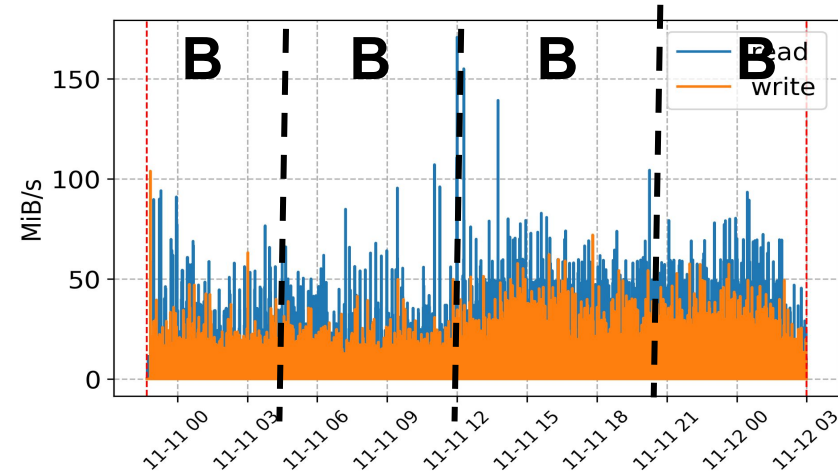
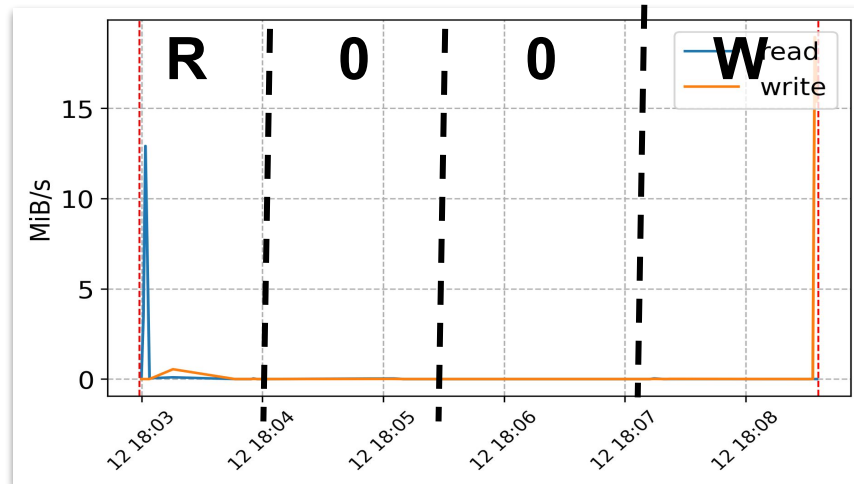
When do applications perform I/O?

- We divide job executions into four equal-sized segments
- Each segment is classified as: **Write (W)**, **Read (R)**, **Both (B)**, or **No I/O (0)**.
- Only segments with bandwidth above 1 MB/s are considered, to reduce noise.

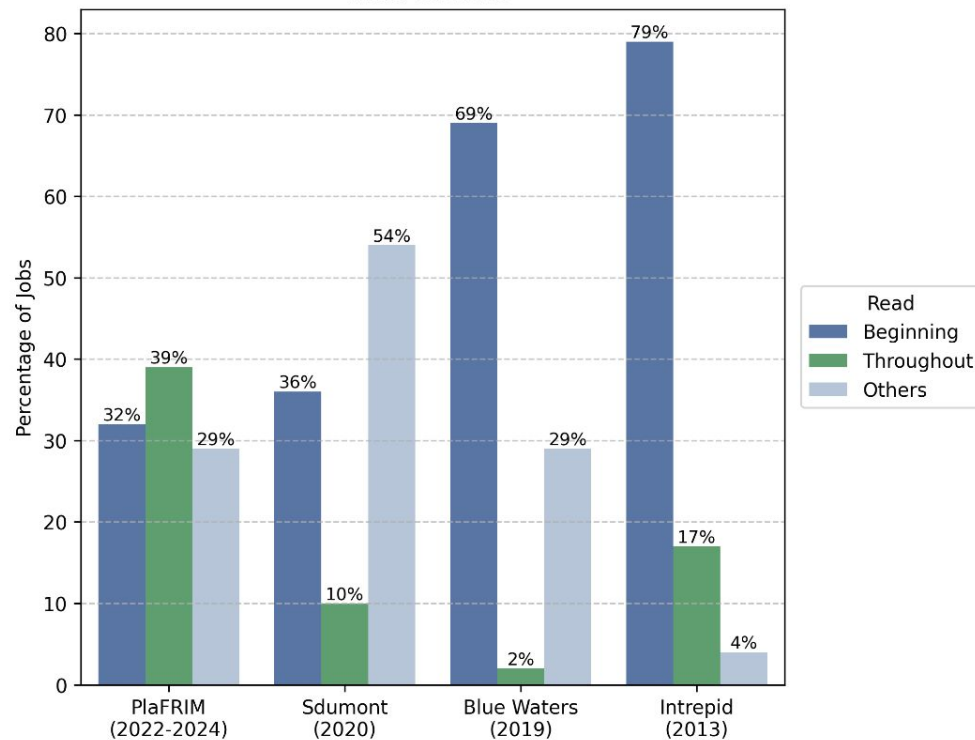


When do applications perform I/O?

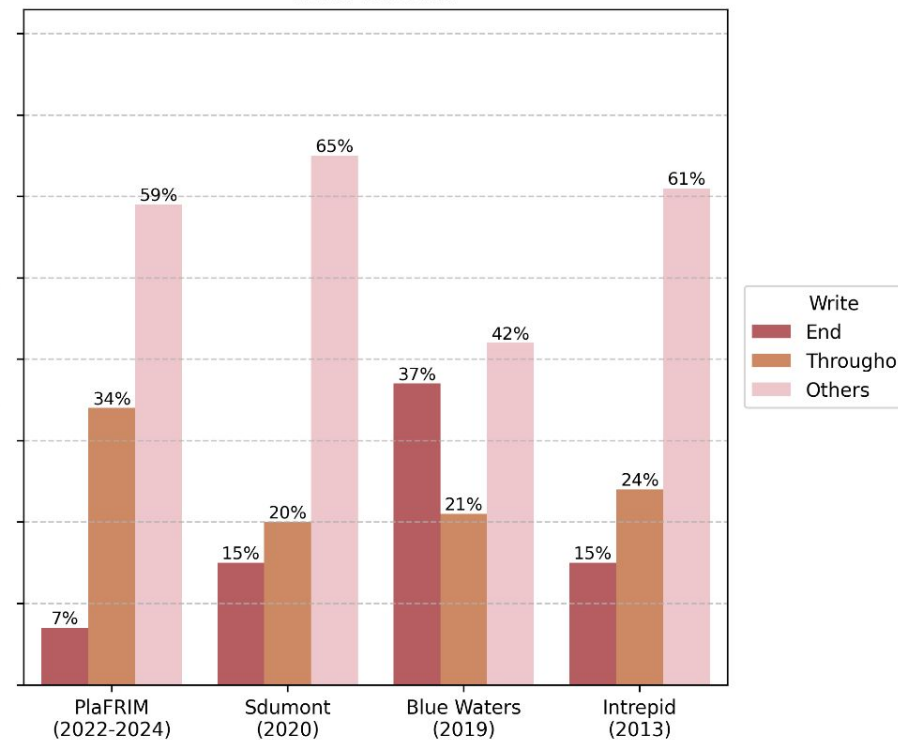
- We divide job executions into four equal-sized segments
- Each segment is classified as: **Write (W)**, **Read (R)**, **Both (B)**, or **No I/O (0)**.
- Only segments with bandwidth above 1 MB/s are considered, to reduce noise.



Read Behavior



Write Behavior



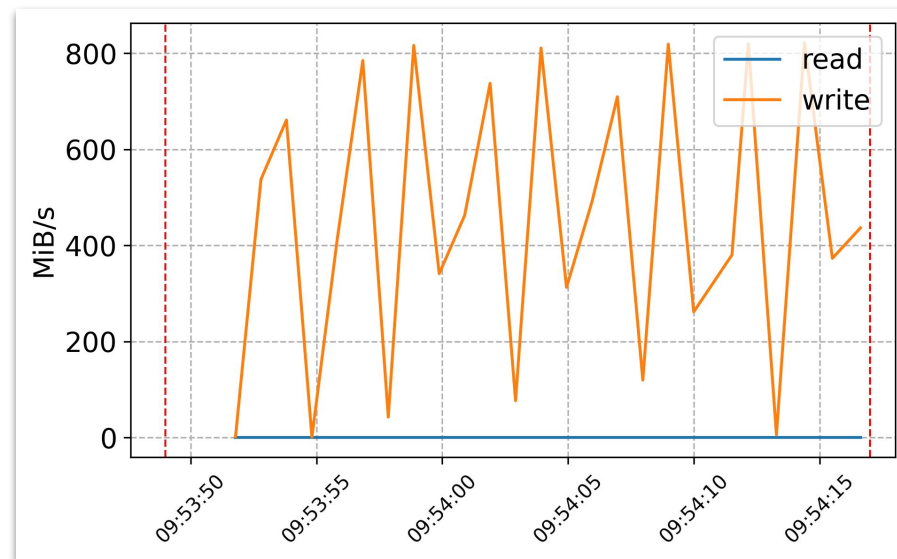
- Most jobs read at the start, especially on older systems or for large jobs.
- Shorter jobs often write at any time.
- Many jobs don't follow standard I/O patterns.
- System-wide I/O tuning should not assume fixed write timing.

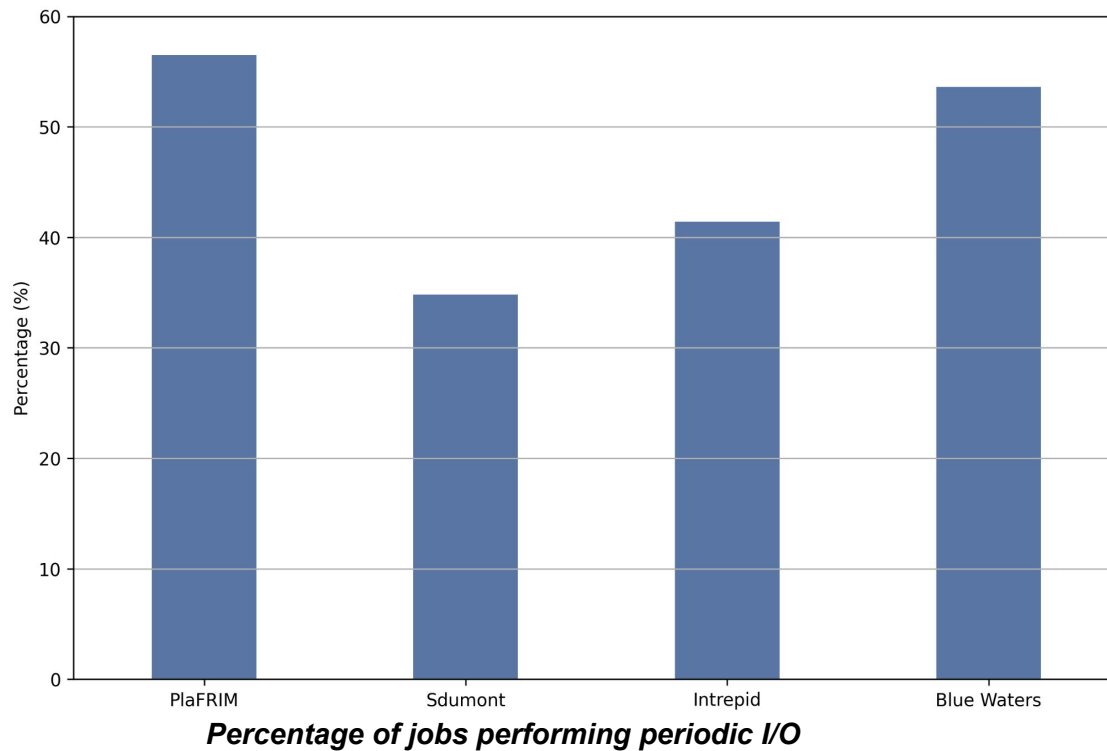
Are applications periodic?

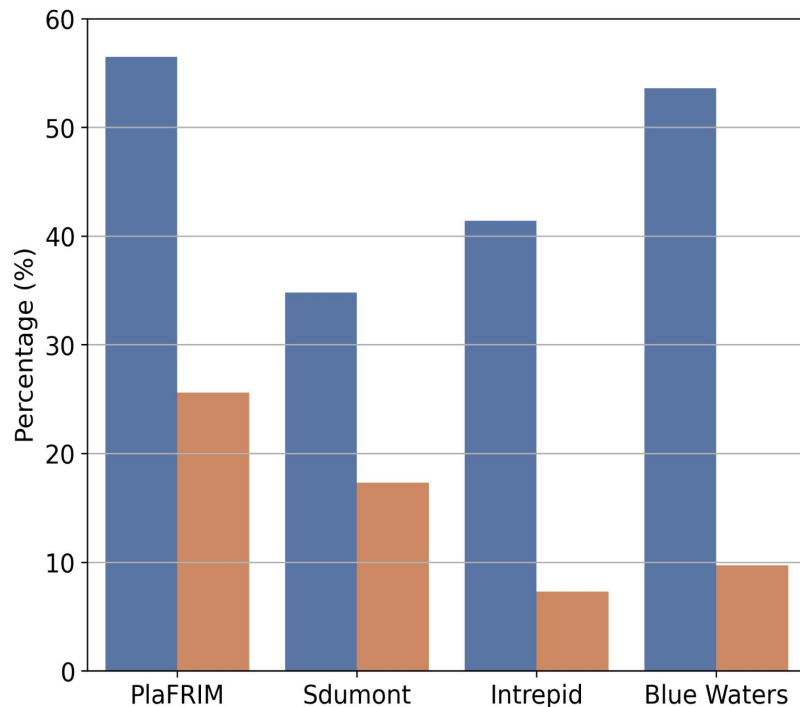
- Periodic I/O is a common assumption in HPC. For instance checkpointing patterns
- Several I/O scheduling approaches rely on this behavior, especially to overlap computation and I/O phases
- We used **FTIO**^[1] to detect and quantify how often jobs perform I/O periodically

1. Tarraf, A., Bandet, A., Boito, F., Pallez, G., & Wolf, F. (2024, May). Capturing periodic I/O using frequency techniques. In 2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS) (pp. 465-478). IEEE.

<https://github.com/tuda-parallel/FTIO/>







Percentage of jobs performing periodic I/O

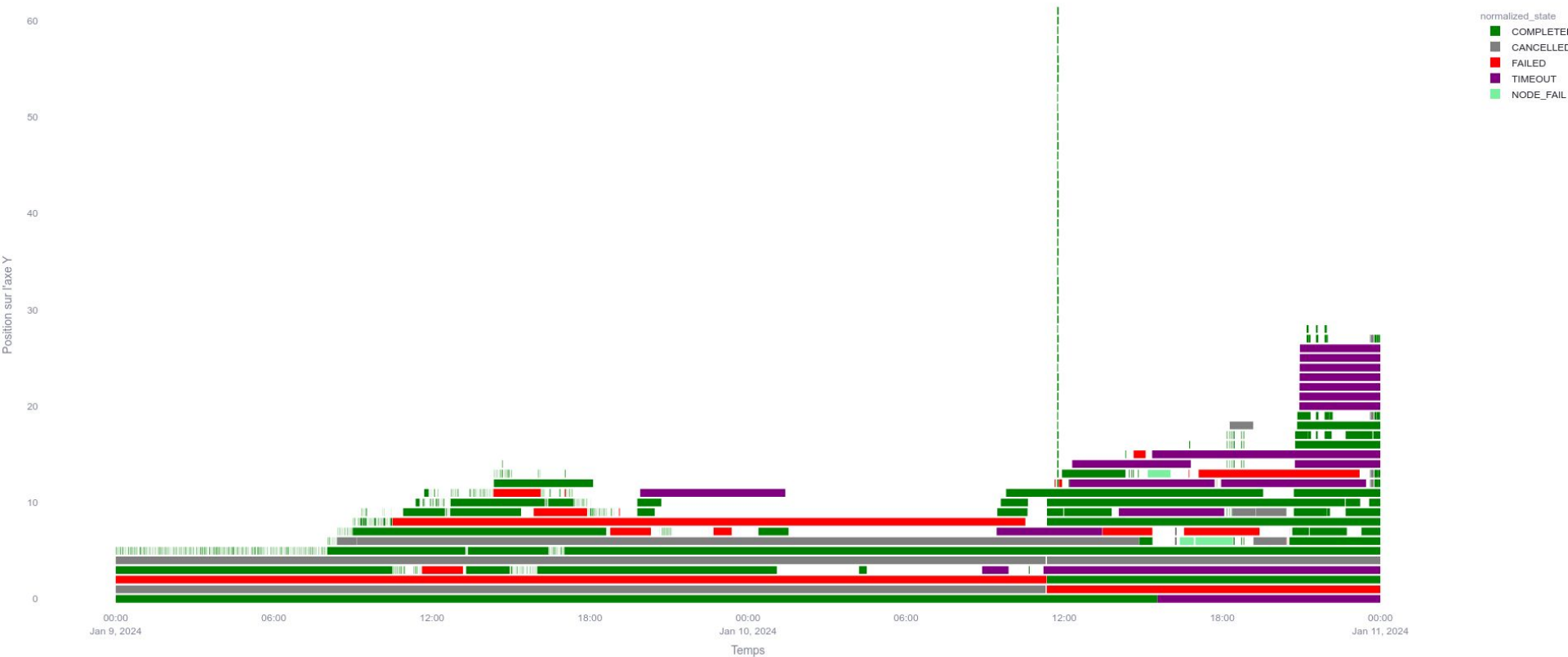
- For jobs >2 phases, less than 26% of jobs perform I/O periodically
- These jobs account for a significant share of I/O time and node-hours.
- Optimizing for periodic I/O can be useful, but most jobs follow other patterns.

How common are concurrent accesses to the shared I/O infrastructure?

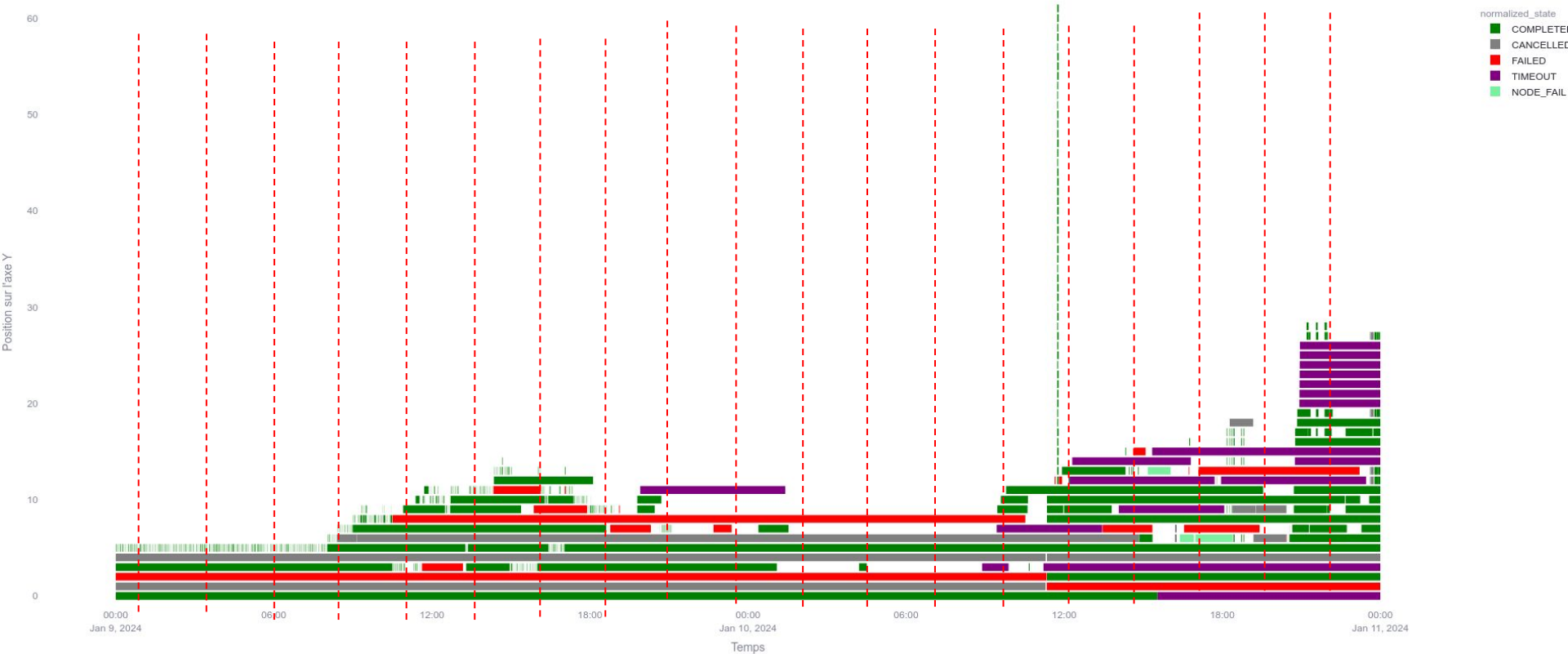
Several approaches to mitigate I/O interference are based on:

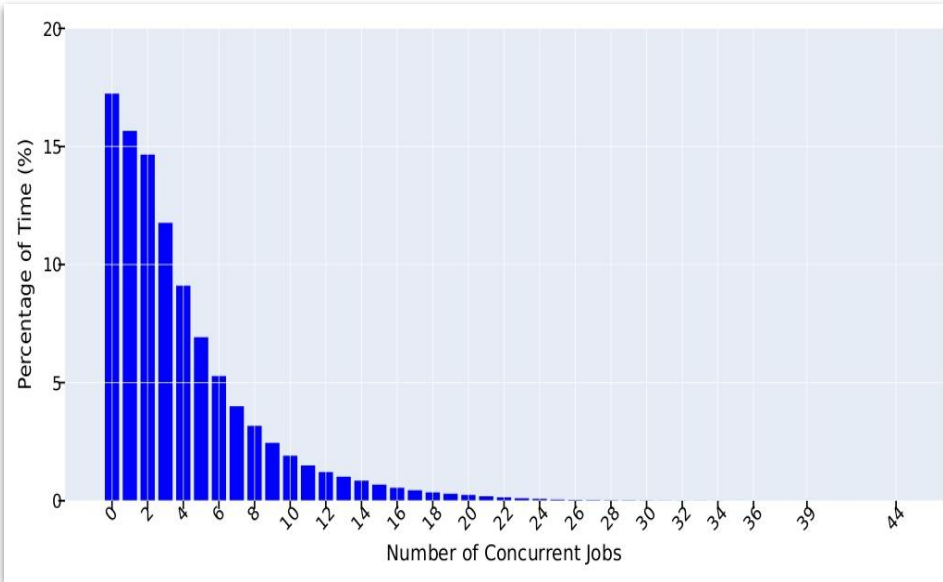
- The assumption that PFSs are often accessed concurrently by multiple applications.
- Concurrent access is a major source of contention
- It impacts global I/O performance.

Overlapping Jobs from 2024-01-09 00:00:00 to 2024-01-11 00:00:00



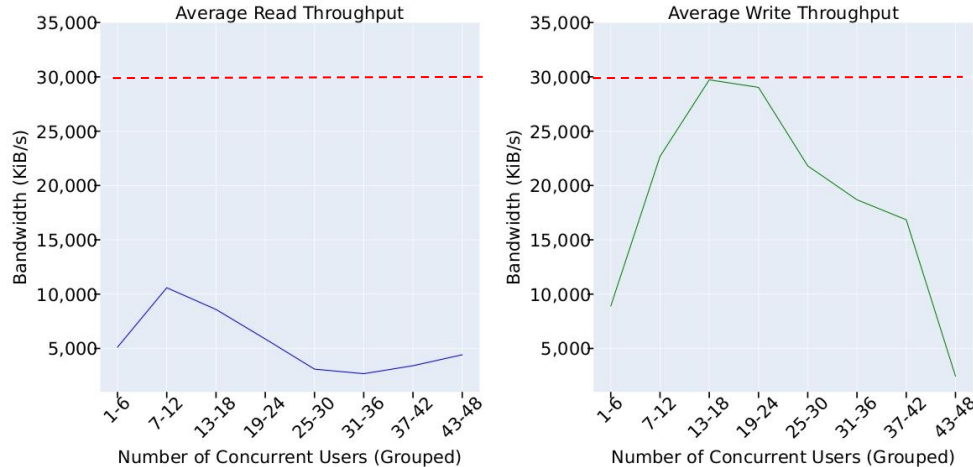
Overlapping Jobs from 2024-01-09 00:00:00 to 2024-01-11 00:00:00





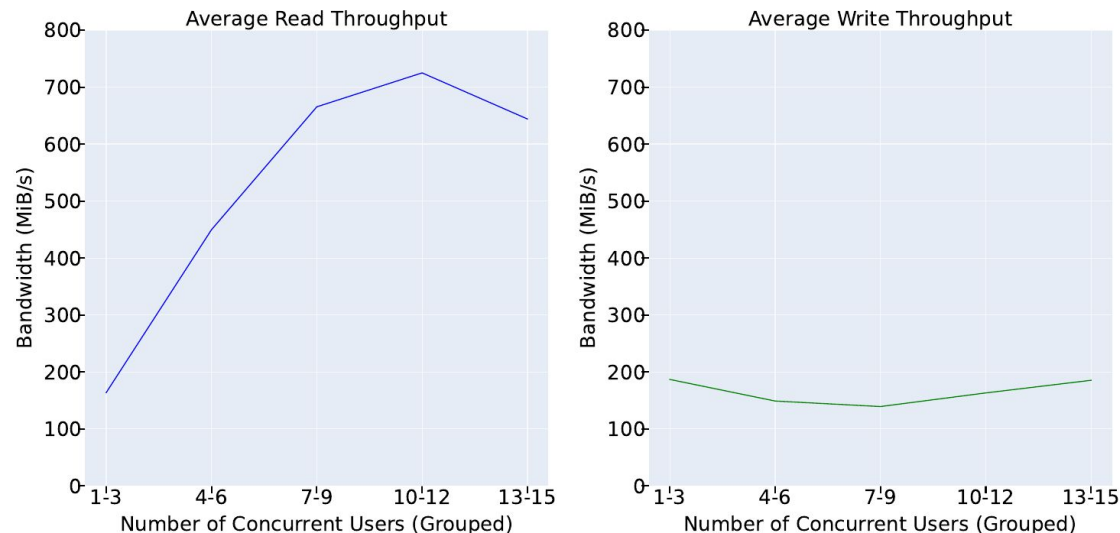
Percentage of time with I/O accesses by concurrent jobs in SDumont
(Similar results for PlaFRIM)

- PFS idle 17% of the time on both platforms
- Single-job access in 16% of the time
- On SDumont, <3% of the time had >15 concurrent I/O jobs



Average bandwidth in SDumont for different degrees of concurrency

- Global bandwidth increases with more applications, up to a limit
- Beyond that, additional applications degrade performance
- For reads, degradation begins well before system capacity (~30 GB/s) is reached



Average bandwidth in PlaFRIM for different degrees of concurrency

- In PlaFRIM, we observe a similar behavior, but for different operations!

How common are concurrent accesses to the shared I/O infrastructure?

- Concurrent I/O is less common than one might expect
- Yet, it degrades global performance even under low load
- How concurrency affects I/O operations seems to depend heavily on the machine's configuration

Conclusion

- Looking at ~400K I/O traces to see how applications behave: nice and cool 😎
 - The extended paper* explores how some of these behaviors can impact I/O optimization strategies
 - And yes, we made the traces available!
- <https://zenodo.org/records/14965920>

* <https://inria.hal.science/hal-04887809>

A Deep Look Into the Temporal I/O Behavior of HPC Applications

Francieli Boito*, Luan Teylo*, Mihail Popov†, Theo Jolivel†, François Tessier†, Jakob Luetigau†, Julien Moumoul†, Ahmad Tarrat†, André Carneiro‡, and Carla Osthoff§

*Univ. Bordeaux, CNRS, Bordeaux INP, Iriria, LaBRI, UMR 5800, F-33400, Talence, France

†Univ. Rennes, Iriria, CNRS, IRISA, Rennes, France

‡Department of Computer Science, Technical University of Darmstadt, Darmstadt, Germany

§LNCX, Petropolis - RJ, 25651-075, Brazil

{first_name}.{last_name}@inria.fr, ahmad.tarrat@tu-darmstadt.de, {andrey.osthoff@lncx.br, {first_name}.{last_name}@inria.fr, ahmad.tarrat@tu-darmstadt.de, {andrey.osthoff@lncx.br

Abstract—The increasing gap between compute and I/O speeds in high-performance computing (HPC) systems imposes the need for techniques to improve applications' I/O performance. Such techniques must rely on assumptions about I/O behavior in order to efficiently allocate I/O resources such as burst buffers, to schedule accesses to the shared parallel file system or to delay certain applications at the batch scheduler level to prevent contention, for instance. In this paper, we verify these common assumptions over 400,000 traces from real HPC systems. By combining traces from diverse systems, we characterize the behaviors observed in real HPC workloads. Among other findings, we show that I/O activity tends to last for a few seconds, and that periodic I/O files are the minority, but responsible for a large portion of the improvement yielded by popular approaches for I/O performance. Furthermore, we make predictions for the expected improvement. Our work provides valuable insights to everyone working to alleviate the I/O bottleneck in HPC.

Index Terms—high-performance computing, parallel file systems, workload characterization, temporal I/O behavior

I. INTRODUCTION

In high-performance computing (HPC) platforms, applications, usually submitted as batch jobs, rely on a parallel file system (PFS) to access persistent data. The PFS is deployed over a set of dedicated resources, separate from compute resources, which are shared by all concurrent jobs. Over the years, the gap between computing performance and the ability of storage systems to store and retrieve data has widened considerably: while supercomputers have become increasingly efficient for computing tasks, storage systems have become relatively less efficient [5]. At the same time, new, non-traditional, HPC applications such as data analytics and ma-

chine learning have shifted the workload toward more data intensity [2]. Therefore, I/O has been an important bottleneck for performance and efficient system usage at scale.

Many techniques have been proposed to improve I/O performance, often focusing on mitigating contention in concurrent accesses. Common examples are the allocation of resources such as burst buffers or I/O nodes [4, 63, 64, 68, 69, 67, 69, 73, 75, 76], scheduling concurrent accesses to the shared PFS [6, 13, 26, 49, 44], adapting applications (by changing when and how much data they write, for example) according to current load [62, 64, 67, 72] or I/O-aware batch scheduling [11, 63, 69, 71]. With rare exceptions, these techniques rely on some assumptions about the workload characteristics. As a result, information about applications' temporal I/O behaviors is an important building block to allow for smart decision-making for I/O optimization.

In this paper, we verify such assumptions by extensively studying the temporal I/O behavior of HPC applications over a decade. While many efforts have described other I/O characteristics such as request size and used I/O API [6, 63, 61, 69, 77], the temporal behavior has been the focus of less work. Moreover, assumptions about it are often based on experience (e.g., periodic writes can be due to checkpointing, which is something applications often need [23, 67]) and on knowledge from specific applications or domains [23, 63, 29, 79]. By studying large sets of traces from real HPC systems, we aim at characterizing the workload we can expect in practice and therefore provide valuable tools for anyone working to improve HPC I/O. For this analysis, we gather traces of I/O activity from four different systems, including both large and small scale systems and facilities, covering a diverse group of user communities. Through this in-depth study, we answer the following questions:

a) **When do applications perform I/O?** While work on resource allocation usually considers steady activity [10, 67, 68], which may waste resources [3, 67], efforts to overlap I/O with compute time (dedicated cores to perform I/O operations [25, use of an ephemeral file system [23]) rely on operations being more sparse, with sufficient compute time in between. While answering this question, we also verify the common belief that HPC applications read at the beginning

This work has received funding from multiple sources. The authors gratefully acknowledge the support from the Exa-SYST project of the NanEPC program (ANR-22-EXPL-0045), as part of the France 2030 initiative, and the NCSA-Irria-ANL-RISC-DC, Iriria-UTM Joint Laboratory of Exascale Computing (JLESC). Furthermore, the authors acknowledge the German Federal Ministry of Education and Research (BMBF) for supporting this work of Science and Research, Air and Climate (IMWK) and the Russian Ministry of Science and Research, Air and Climate (IMWK) for supporting this work of EXASCALE and the European Commission under the Exascale Programme ADMBE (GA No. 995744, BMBF funding No. 10HPC006). This work was also conducted within the Exascale Joint team between Iriria and LNCX.